

```
!pip install yfinance statsmodels scikit-learn matplotlib seaborn joblib
```

```
Requirement already satisfied: yfinance in /usr/local/lib/python3.12/dist-packages (0.2.66)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.12/dist-packages (0.14.6)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.3)
Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.2.2)
Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.0.2)
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.32.4)
Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.0.13)
Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.10.0)
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2025.2)
Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.4.7)
Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.0.6)
Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.13.5)
Requirement already satisfied: curl_cffi>=0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.15.0)
Requirement already satisfied: protobuf>=3.19.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (5.29.6)
Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (15.0.1)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in /usr/local/lib/python3.12/dist-packages (from statsmodels) (1.16.3)
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.12/dist-packages (from statsmodels) (1.0.2)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.12/dist-packages (from statsmodels) (26.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4>=4.11.1->yfinance) (2.5.0)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4>=4.11.1->yfinance) (4.12.0)
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (2.0.0)
Requirement already satisfied: certifi>=2024.2.2 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (2024.8.30)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (13.9.4)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.3.0->yfinance) (2025.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinance) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinance) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinance) (2.3.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=2.0.0->curl_cffi>=0.7->yfinance) (2.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->curl_cffi>=0.7->yfinance) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->curl_cffi>=0.7->yfinance) (2.18.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->curl_cffi>=0.7->yfinance) (0.1.2)
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import yfinance as yf
from statsmodels.tsa.stattools import adfuller, kpss
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.seasonal import seasonal_decompose
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import joblib
import os
import json
from datetime import datetime, timedelta

TICKER = "AAPL"
START_DATE = "2020-01-01"
END_DATE = datetime.today().strftime("%Y-%m-%d")
TEST_SIZE = 60
FORECAST_DAYS = 30
SAVE_DIR = "arima_artifacts"
os.makedirs(SAVE_DIR, exist_ok=True)
```

```
raw = yf.download(TICKER, start=START_DATE, end=END_DATE, progress=False)

if raw.empty:
    raise ValueError(f"No data returned for ticker '{TICKER}'. Check the symbol.")

# Flatten MultiIndex columns that yfinance sometimes returns
if isinstance(raw.columns, pd.MultiIndex):
    raw.columns = raw.columns.get_level_values(0)

df = raw[["Open", "High", "Low", "Close", "Volume"]].copy()
df.index = pd.to_datetime(df.index)
df.sort_index(inplace=True)
```

```
print(f"    ✓ Downloaded {len(df):,} trading days  ({df.index[0].date()} → {df.index[-1].date()})\n")
print(df.tail(5))
```

```
/tmp/ipykernel_3725/1187957322.py:1: FutureWarning: YF.download() has changed argument auto_adjust default to True
  raw = yf.download(TICKER, start=START_DATE, end=END_DATE, progress=False)
    ✓ Downloaded 1,619 trading days  (2020-01-02 → 2026-06-11)
```

Price Date	Open	High	Low	Close	Volume
2026-06-05	312.859985	315.170013	307.149994	307.339996	65310500
2026-06-08	308.739990	317.399994	301.170013	301.540009	77949100
2026-06-09	300.279999	300.750000	287.779999	290.549988	70108800
2026-06-10	290.739990	294.750000	287.380005	291.579987	52793300
2026-06-11	293.720001	297.000000	289.589996	295.630005	42523300

```
print("\n— Descriptive Statistics (Close Price) —")
print(df["Close"].describe().round(2))
missing = df.isnull().sum()
print(f"\Missing Values \n{missing}")
df.fillna(inplace=True)
df["Returns"] = df["Close"].pct_change()
df["MA_20"] = df["Close"].rolling(20).mean()
df["MA_50"] = df["Close"].rolling(50).mean()
df["Volatility"] = df["Returns"].rolling(20).std()
df["Log_Close"] = np.log(df["Close"])

fig, axes = plt.subplots(3, 2, figsize=(16, 14))
fig.suptitle(f"{TICKER} – Exploratory Data Analysis", fontsize=16, fontweight="bold")

# Plot 1 – Closing price + MAs
ax = axes[0, 0]
ax.plot(df.index, df["Close"], label="Close", linewidth=1.2, color="#2196F3")
ax.plot(df.index, df["MA_20"], label="MA 20", linewidth=1, color="#FF9800", linestyle="--")
ax.plot(df.index, df["MA_50"], label="MA 50", linewidth=1, color="#E91E63", linestyle="--")
ax.set_title("Closing Price & Moving Averages"); ax.set_ylabel("Price (USD)")
ax.legend(fontsize=8); ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))

# Plot 2 – Volume
ax = axes[0, 1]
ax.bar(df.index, df["Volume"], color="#90CAF9", width=1, alpha=0.7)
ax.set_title("Trading Volume"); ax.set_ylabel("Volume")
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))

# Plot 3 – Daily returns
ax = axes[1, 0]
ax.plot(df.index, df["Returns"], color="#4CAF50", linewidth=0.8)
ax.axhline(0, color="black", linewidth=0.8, linestyle="--")
ax.set_title("Daily Returns"); ax.set_ylabel("Return (%)")
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))

# Plot 4 – Return distribution
ax = axes[1, 1]
df["Returns"].dropna().hist(bins=60, ax=ax, color="#7E57C2", edgecolor="white")
ax.set_title("Return Distribution"); ax.set_xlabel("Daily Return")

# Plot 5 – Rolling volatility
ax = axes[2, 0]
ax.plot(df.index, df["Volatility"], color="#F44336", linewidth=1)
ax.set_title("20-Day Rolling Volatility"); ax.set_ylabel("Std Dev of Returns")
ax.xaxis.set_major_formatter(mdates.DateFormatter("%Y"))

# Plot 6 – Correlation heatmap
ax = axes[2, 1]
corr = df[["Open", "High", "Low", "Close", "Volume", "Returns", "Volatility"]].corr()
sns.heatmap(corr, ax=ax, annot=True, fmt=".2f", cmap="coolwarm",
            linewidths=0.5, annot_kws={"size": 7})
ax.set_title("Feature Correlation Heatmap")

plt.tight_layout()
eda_path = os.path.join(SAVE_DIR, "eda_plots.png")
plt.savefig(eda_path, dpi=150, bbox_inches="tight")
plt.close()
```

```
— Descriptive Statistics (Close Price) —
count    1619.00
mean      171.93
std        55.16
min        54.16
25%       134.13
50%       168.04
75%       212.40
max       315.20
```

```
Name: Close, dtype: float64
\Missing Values
Price
Open      0
High      0
Low       0
Close     0
Volume    0
dtype: int64
<>:4: SyntaxWarning: invalid escape sequence '\M'
<>:4: SyntaxWarning: invalid escape sequence '\M'
/tmp/ipykernel_3725/3234255004.py:4: SyntaxWarning: invalid escape sequence '\M'
print(f"\Missing Values \n{missing}")
```

```
series = df["Close"].dropna()

def adf_test(ts, label="Series"):
    """Augmented Dickey-Fuller test."""
    result = adfuller(ts, autolag="AIC")
    stat, p, _, _, crit, _ = result
    is_stationary = p < 0.05
    print(f"\n — ADF Test : {label} —")
    print(f"  ADF Statistic : {stat:.4f}")
    print(f"  p-value       : {p:.4f}")
    for k, v in crit.items():
        print(f"    Critical ({k}) : {v:.4f}")
    print(f"  → {'✓ STATIONARY' if is_stationary else '✗ NON-STATIONARY'} (p {'<' if is_stationary else '≥'} 0.05)")
    return is_stationary

stationary = adf_test(series, label="Close Price")

diff_order = 0
series_diff = series.copy()
while not stationary and diff_order < 3:
    diff_order += 1
    series_diff = series_diff.diff().dropna()
    stationary = adf_test(series_diff, label=f"Differenced (d={diff_order})")

print(f"\n  ✓ Using d = {diff_order} for ARIMA\n")
```

```
— ADF Test : Close Price —
ADF Statistic : -0.7304
p-value       : 0.8387
Critical (1%) : -3.4344
Critical (5%) : -2.8633
Critical (10%) : -2.5677
→ ✗ NON-STATIONARY (p ≥ 0.05)

— ADF Test : Differenced (d=1) —
ADF Statistic : -39.6507
p-value       : 0.0000
Critical (1%) : -3.4344
Critical (5%) : -2.8633
Critical (10%) : -2.5677
→ ✓ STATIONARY (p < 0.05)

✓ Using d = 1 for ARIMA
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 4))
plot_acf(series_diff, lags=40, ax=axes[0], title="ACF (Differenced Series)", color="#2196F3")
plot_pacf(series_diff, lags=40, ax=axes[1], title="PACF (Differenced Series)", color="#E91E63", method="ywmm")
plt.tight_layout()
acf_path = os.path.join(SAVE_DIR, "acf_pacf.png")
plt.savefig(acf_path, dpi=150, bbox_inches="tight")
plt.close()
print(f"  ✓ ACF/PACF plots saved → {acf_path}")

train_series = series[:-TEST_SIZE] # Use only training data for order selection

best_aic = np.inf
best_order = (1, diff_order, 1) # Sensible default fallback

for p in range(0, 4):
    for q in range(0, 4):
        try:
            model_tmp = ARIMA(train_series, order=(p, diff_order, q))
```

```

        result_tmp = model_tmp.fit()
        if result_tmp.aic < best_aic:
            best_aic = result_tmp.aic
            best_order = (p, diff_order, q)
    except Exception:
        continue

print(f"    ✓ Best ARIMA order : {best_order} (AIC = {best_aic:.2f})\n")

    ✓ ACF/PACF plots saved → arima_artifacts/acf_pacf.png
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provide
self._init_dates(dates, freq)

```

```

train = series[:-TEST_SIZE]
test = series[-TEST_SIZE:]
print(f"    Train size : {len(train):,} | Test size : {len(test):,}")

arima_model = ARIMA(train, order=best_order)
arima_result = arima_model.fit()

print("\n— Model Summary (truncated) —")
print(arima_result.summary().tables[0]) # Print just the top info table
print()

fig = arima_result.plot_diagnostics(figsize=(14, 8))
fig.suptitle(f"ARIMA{best_order} – Residual Diagnostics", fontsize=13, fontweight="bold")
diag_path = os.path.join(SAVE_DIR, "residual_diagnostics.png")
plt.savefig(diag_path, dpi=150, bbox_inches="tight")
plt.close()
print(f"    ✓ Residual diagnostics saved → {diag_path}\n")

```

```

Train size : 1,559 | Test size : 60

— Model Summary (truncated) —
                                SARIMAX Results
=====
Dep. Variable:                  Close    No. Observations:              1559
Model:                        ARIMA(0, 1, 0)    Log Likelihood                -3941.736
Date:                        Fri, 12 Jun 2026    AIC                          7885.473
Time:                        06:01:55          BIC                          7890.824
Sample:                      0              HQIC                       7887.463
                                - 1559
Covariance Type:              opg
=====

/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided,
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided,
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided,
self._init_dates(dates, freq)
✓ Residual diagnostics saved → arima_artifacts/residual_diagnostics.png

```

```

# — Rolling one-step-ahead forecast on test set —————
history      = list(train)
predictions  = []

for t in range(len(test)):
    model_step = ARIMA(history, order=best_order)
    result_step = model_step.fit()
    yhat        = result_step.forecast(steps=1)[0]
    predictions.append(yhat)
    history.append(test.iloc[t]) # Append actual observation

predictions = np.array(predictions)
actuals     = test.values

mae = mean_absolute_error(actuals, predictions)
rmse = np.sqrt(mean_squared_error(actuals, predictions))
mape = np.mean(np.abs((actuals - predictions) / actuals)) * 100
r2 = r2_score(actuals, predictions)

print(f" MAE   : {mae:.4f}")
print(f" RMSE  : {rmse:.4f}")
print(f" MAPE  : {mape:.2f} %")
print(f" R²    : {r2:.4f}")

```

```

MAE   : 3.1894
RMSE  : 4.1105
MAPE  : 1.14 %
R²    : 0.9645

```

```

final_model = ARIMA(series, order=best_order)
final_result = final_model.fit()

forecast_obj = final_result.get_forecast(steps=FORECAST_DAYS)
forecast_mean = forecast_obj.predicted_mean
forecast_ci   = forecast_obj.conf_int(alpha=0.05) # 95% confidence interval

last_date    = series.index[-1]
forecast_dates = pd.bdate_range(start=last_date + timedelta(days=1), periods=FORECAST_DAYS)
forecast_series = pd.Series(forecast_mean.values, index=forecast_dates)
ci_lower       = pd.Series(forecast_ci.iloc[:, 0].values, index=forecast_dates)
ci_upper       = pd.Series(forecast_ci.iloc[:, 1].values, index=forecast_dates)

fig, axes = plt.subplots(2, 1, figsize=(16, 12))
fig.suptitle(f"ARIMA{best_order} – {TICKER} Stock Price Prediction", fontsize=15, fontweight="bold")

ax = axes[0]
ax.plot(test.index, actuals, label="Actual", color="#2196F3", linewidth=2)
ax.plot(test.index, predictions, label="Predicted", color="#FF5722", linewidth=1.8, linestyle="--")
ax.set_title(f"Test Set : Actual vs Predicted (MAE={mae:.2f}, RMSE={rmse:.2f}, MAPE={mape:.2f}%)")
ax.set_ylabel("Price (USD)"); ax.legend()
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))

ax = axes[1]
lookback = 120 # Show last N days of history for context
ax.plot(series.index[-lookback:], series.values[-lookback:],
        label="Historical Close", color="#2196F3", linewidth=1.8)

```

```

ax.plot(forecast_series.index, forecast_series.values,
        label=f"Forecast ({FORECAST_DAYS} days)", color="#4CAF50", linewidth=2, marker="o", markersize=3)
ax.fill_between(forecast_series.index, ci_lower, ci_upper,
                alpha=0.25, color="#4CAF50", label="95% Confidence Interval")
ax.axvline(x=series.index[-1], color="grey", linestyle="--", linewidth=1, label="Forecast Start")
ax.set_title(f"Future {FORECAST_DAYS}-Day Price Forecast")
ax.set_ylabel("Price (USD)"); ax.legend()
ax.xaxis.set_major_formatter(mdates.DateFormatter("%b %Y"))

```

```

plt.tight_layout()
forecast_path = os.path.join(SAVE_DIR, "forecast_plot.png")
plt.savefig(forecast_path, dpi=150, bbox_inches="tight")
plt.close()
print(f"    ✓ Forecast plot saved → {forecast_path}\n")

```

```

print("— Forecasted Prices —")
forecast_df = pd.DataFrame({
    "Date"      : forecast_series.index,
    "Forecast"  : forecast_series.values.round(2),
    "Lower_95"  : ci_lower.values.round(2),
    "Upper_95"  : ci_upper.values.round(2),
})
print(forecast_df.to_string(index=False))
forecast_df.to_csv(os.path.join(SAVE_DIR, "forecast_values.csv"), index=False)
print(f"\n    ✓ Forecast CSV saved → {SAVE_DIR}/forecast_values.csv\n")

```

```

/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided,
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided,
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided,
self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available
return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index is available
return get_prediction_index(
    ✓ Forecast plot saved → arima_artifacts/forecast_plot.png

```

— Forecasted Prices —

Date	Forecast	Lower_95	Upper_95
2026-06-12	295.63	289.59	301.67
2026-06-15	295.63	287.08	304.18
2026-06-16	295.63	285.16	306.10
2026-06-17	295.63	283.54	307.72
2026-06-18	295.63	282.11	309.15
2026-06-19	295.63	280.82	310.44
2026-06-22	295.63	279.64	311.62
2026-06-23	295.63	278.53	312.73
2026-06-24	295.63	277.50	313.76
2026-06-25	295.63	276.52	314.74
2026-06-26	295.63	275.58	315.68
2026-06-29	295.63	274.69	316.57
2026-06-30	295.63	273.84	317.42
2026-07-01	295.63	273.01	318.25
2026-07-02	295.63	272.22	319.04
2026-07-03	295.63	271.45	319.81
2026-07-06	295.63	270.71	320.55
2026-07-07	295.63	269.98	321.28
2026-07-08	295.63	269.28	321.98
2026-07-09	295.63	268.60	322.66
2026-07-10	295.63	267.93	323.33
2026-07-13	295.63	267.28	323.98
2026-07-14	295.63	266.64	324.62
2026-07-15	295.63	266.02	325.24
2026-07-16	295.63	265.41	325.85
2026-07-17	295.63	264.81	326.45
2026-07-20	295.63	264.22	327.04
2026-07-21	295.63	263.65	327.61
2026-07-22	295.63	263.08	328.18
2026-07-23	295.63	262.52	328.74

✓ Forecast CSV saved → arima_artifacts/forecast_values.csv

```

model_path = os.path.join(SAVE_DIR, "arima_model.pkl")
joblib.dump(final_result, model_path)
print(f"    ✓ Model saved      → {model_path}")

```

✓ Model saved → arima_artifacts/arima_model.pkl

